

Prática Offline 3

1. [TAREFA] Simulação de Sistema de *Streaming*: Cliente-Servidor.

Partindo da prática offline 2, esta simulação modela o processo de autenticação e recuperação de conteúdo, focando na eficiência de busca (lado do cliente) e na organização de grandes volumes de dados (lado do servidor).

- **O lado do cliente (*Frontend/App*).**
 - O cliente atua como a interface de consumo.
 - Para evitar requisições desnecessárias à rede (que possuem custo de latência e consumo de banda), ele utiliza um cache local.
 - Estruturas: tabela hash + lista autoajustável.
 - Função: Armazenar títulos acessados recentemente, metadados de login e outras informações frequentemente utilizadas pelo usuário.
 - Por que usar?
 - A tabela hash permite localizar rapidamente um item armazenado, com tempo médio de busca $O(1)$, enquanto a lista autoajustável mantém a ordem de utilização dos elementos.
 - Dessa forma, o sistema consegue acessar dados de forma eficiente e identificar quais itens foram utilizados mais recentemente.
 - *Eviction* (Esvaziamento):
 - Quando o cache atinge sua capacidade máxima, a política LRU (*Least Recently Used*) deve ser aplicada. O item menos recentemente utilizado é removido da estrutura para liberar espaço para novos dados, simulando o gerenciamento de memória encontrado em dispositivos reais, como smartphones, Smart TVs e aplicações de *streaming*.
 - Como fazer?
 - Trata-se de uma decisão de projeto amplamente utilizada em sistemas reais de cache, pois combina acesso rápido aos dados com uma política eficiente de substituição de elementos.
 - Registro de preferências e recomendação.
 - Estrutura: árvore de difusão (*splay*).
 - Função: Registrar o histórico recente de consumo do usuário e inferir suas preferências dinâmicas para geração de recomendações.
 - Por que usar?
 - A árvore *splay* é uma estrutura autoajustável que reorganiza seus nós com base nos acessos realizados. Sempre que um filme ou categoria é acessado pelo usuário, seu nó é movido para próximo da raiz da árvore.
 - Dessa forma, os elementos mais recentemente consumidos permanecem em posições privilegiadas, refletindo o interesse atual do usuário.
 - Como o comportamento de consumo em plataformas de *streaming* apresenta forte localidade temporal, ou seja, usuários tendem a assistir conteúdos semelhantes em curtos períodos de tempo, a *splay* adapta continuamente sua estrutura para representar essas

- preferências.
 - Sistema de recomendação.
 - Com base na árvore *splay*, o sistema pode identificar:
 - A categoria ou filme atualmente mais relevante (raiz da árvore);
 - Interesses secundários (subárvores próximas à raiz);
 - Mudanças recentes no padrão de consumo.
 - Essas informações permitem ao sistema gerar recomendações dinâmicas, como:
 - Sugestão de filmes da mesma categoria do nó raiz;
 - Indicação de conteúdos similares aos nós mais frequentemente acessados;
 - Ajuste contínuo das recomendações conforme o comportamento do usuário.
- O lado do servidor (*Backend/Database*).
 - O servidor detém o "catálogo mestre".
 - Ele precisa lidar com uma quantidade massiva de títulos e simular a persistência em um meio físico.
 - Armazenamento físico: lista ligada.
 - Função: Simula a disposição sequencial ou encadeada dos dados em um disco ou memória persistente.
 - Característica: É uma estrutura linear onde cada elemento aponta para o próximo. Por si só, a busca aqui seria lenta ($O(n)$).
 - Indexação: tabela hash.
 - Função: Atua como um "mapa de endereços" para a lista ligada.
 - Por que usar?
 - A tabela hash permite localizar rapidamente um filme a partir do seu identificador, reduzindo o custo médio de busca para $O(1)$.
 - Cada entrada da tabela aponta diretamente para uma referência do nó correspondente na lista ligada, eliminando a necessidade de percorrer toda a estrutura de armazenamento.
 - Caso opte pelo tratamento de colisões baseado em encadeamento, use algum método de autoajuste nas listas de colisão. Justifique.
 - Análise de popularidade e acessos: árvore *splay*.
 - Função: Registrar e reorganizar dinamicamente a frequência de acesso aos filmes no sistema como um todo, permitindo análise de popularidade global.
 - Sistema de recomendação e análise global.
 - Com base na árvore *splay*, o servidor pode:
 - Identificar os filmes mais populares da plataforma (próximos da raiz);
 - Detectar tendências de acesso em tempo real;
 - Auxiliar na geração de rankings dinâmicos de conteúdos;
 - Apoiar sistemas de recomendação baseados em popularidade global.
- Camada de comunicação (Cliente ↔ Servidor).
 - A comunicação entre cliente e servidor ocorre por meio da troca de mensagens contendo requisições, respostas, informações de autenticação e metadados dos filmes.
 - Compressão de mensagens.
 - Estrutura: árvore de huffman.
 - Função: Comprimir e descomprimir as mensagens transmitidas pela "rede".
 - Por que usar?
 - O algoritmo de huffman realiza compressão sem perdas, atribuindo códigos menores aos caracteres mais frequentes e códigos maiores aos menos frequentes. Isso reduz a quantidade de dados transmitidos sem alterar o conteúdo original da mensagem.
 - Funcionamento.
 - Antes de enviar uma mensagem, o emissor (cliente ou servidor) utiliza a árvore de huffman para gerar uma versão comprimida dos dados. Ao receber a mensagem, o

destinatário reconstrói a árvore e realiza a descompressão, recuperando exatamente a informação original.

- Exemplos de mensagens.

None

LOGIN_OK

GET /filme/505

FILME:505|Matrix|1999

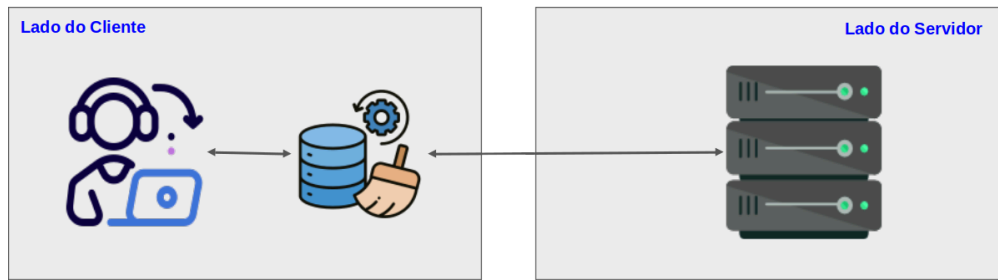
RECOMENDACAO:202|Interestelar

- Benefícios.
 - Redução do volume de dados trafegados;
 - Menor consumo de banda de rede;
 - Menor tempo de transmissão;
 - Preservação integral das informações;
 - Aplicação prática de árvores binárias em sistemas distribuídos.
- Objetivo na simulação.
 - A árvore de huffman é utilizada para otimizar a comunicação entre cliente e servidor, demonstrando como estruturas de dados podem ser empregadas para reduzir custos de transmissão e melhorar a eficiência da troca de informações em sistemas de *streaming*.
- Fluxo da simulação.
 - Requisição: O cliente tenta acessar o título "Matrix".
 - Verificação de cache (tabela hash + LRU).
 - O sistema verifica primeiro o cache local utilizando a tabela hash.
 - Hit: O filme é encontrado no cache e exibido imediatamente.
 - Além disso, o nó correspondente é movido para o início da lista autoajustável (LRU), marcando-o como o item mais recentemente utilizado.
 - Miss: O filme não está presente no cache local. O cliente então dispara uma requisição ao servidor.
 - Processamento no servidor (tabela hash + lista ligada + *splay*).
 - O servidor recebe o identificador do filme e utiliza a tabela hash como índice principal para localizar rapidamente o registro.
 - A tabela hash aponta diretamente para uma referência do nó correspondente na lista ligada, onde os dados completos do filme estão armazenados.
 - Após localizar o filme:
 - O servidor retorna os dados ao cliente.
 - A árvore *splay* do servidor é atualizada, movendo o filme acessado para próximo da raiz, refletindo sua popularidade global.
 - Resposta e atualização no cliente.
 - O servidor envia os dados completos do filme ao cliente.
 - O cliente então:
 - Exibe o conteúdo ao usuário;
 - Insere o filme no cache local;
 - Adiciona o nó ao início da lista LRU;
 - Atualiza sua árvore *splay* de preferências, registrando o acesso ao filme ou à categoria associada.
 - Sugere o próximo filme que o cliente pode acessar (sistema de recomendação).

- Caso o cache atinja sua capacidade máxima, o último elemento da lista LRU (menos recentemente utilizado) é removido automaticamente.
- **Exemplo: O fluxo com IDs (melhor cenário).**
 - Login:
 - O servidor envia ao cliente um catálogo simplificado:
 - [505, Matrix].
 - [101, Avatar].
 - [202, Interestelar].
 - Busca:
 - O usuário escolhe "Matrix", cujo ID é 505. Aqui, o programa deve receber como entrada é o nome do filme. Seu programa deve transformá-lo no ID correspondente.
 - Cache (tabela hash + LRU + *splay* do cliente).
 - O sistema consulta o cache local:
 - `cache.get(505)`.
 - *Hit*: exibe imediatamente e atualiza a LRU e a *splay* de preferências.
 - *Miss*: continua o fluxo para o servidor.
 - Requisição ao servidor.
 - `GET /filme/505`.
 - Servidor (tabela hash + lista ligada + *splay* do servidor).
 - A tabela hash localiza o ID 505.
 - O "ponteiro" leva diretamente ao nó na lista ligada.
 - Os dados do filme são recuperados.
 - A árvore *splay* do servidor é atualizada, movendo o nó 505 para perto da raiz (indicando alta popularidade).
 - Resposta.
 - O servidor retorna os dados completos do filme.
 - O cliente:
 - Exibe o filme;
 - Atualiza o cache (tabela hash + LRU);
 - Atualiza árvore *splay* de preferências do usuário.
- **Resultados esperados.**
 - A simulação passa a representar corretamente três camadas de comportamento:
 - Cliente:
 - Cache eficiente com tabela hash + LRU.
 - Registro de preferências com árvore *splay*.
 - Servidor:
 - Persistência simulada com lista ligada.
 - Acesso rápido com Hash.
 - Popularidade global com árvore *splay*.
 - Comunicação.
 - Simulação de transmissão de dados usando compressão de huffman.
- **Execução da simulação.**
 - Ao iniciar o programa:
 - Devem ser inseridos 1000 filmes na base de dados do servidor.
 - Cada filme deve possuir ID, título, categoria, ano e sinopse.
 - Os registros devem ser armazenados na lista ligada do servidor.
 - A tabela hash do servidor deve ser construída apontando para os respectivos nós da lista.
 - O cache do cliente deve possuir capacidade para armazenar 50 registros.
 - Para simular acessos anteriores, 50 filmes devem ser previamente carregados no cache (tabela hash + lista LRU).
 - As árvores *splay* de preferências do cliente e de popularidade do servidor devem iniciar vazias.

- Após a carga inicial, a interface do cliente deve ser apresentada.
 - Para facilitar a navegação, os filmes podem ser organizados por categorias, por exemplo:
 - Ação.
 - Ficção Científica.
 - Drama.
 - Terror.
 - Comédia.
 - Suspense.
 - Entre outros.
- Conjunto de consultas.
 - Considere a existência de três clientes (que podem ser cadastrados previamente).
 - Para cada cliente, execute métodos que realizem 20 consultas.
 - Distribuição sugerida:
 - 2 consultas inválidas.
 - Consultar IDs inexistentes.
 - 6 consultas com cache *hit*.
 - Consultar filmes previamente armazenados no cache.
 - 6 consultas sem indexação.
 - Desabilitar temporariamente a tabela hash do servidor.
 - Realizar busca sequencial diretamente na lista ligada.
 - 6 consultas com indexação.
 - Realizar as mesmas consultas utilizando a tabela hash do servidor.
 - O programa deve contabilizar o número de comparações em cada situação.
 - Analisar:
 - Comportamento do cache LRU.
 - Ao final da execução:
 - Exibir os 10 filmes mais recentemente utilizados.
 - Mostrar quais registros foram removidos do cache devido à política LRU.
 - Árvore *splay* de preferências do cliente.
 - Após as consultas:
 - Exibir a raiz da árvore.
 - Exibir os cinco filmes ou categorias mais acessados pelos usuários.
 - Árvore *splay* de popularidade do servidor.
 - Após todas as consultas:
 - Exibir os dez filmes mais próximos da raiz.
 - Identificar os conteúdos mais populares globalmente.
 - Compressão de huffman.
 - Selecionar algumas mensagens trocadas entre cliente e servidor.
 - Para cada mensagem:
 - Mostrar tamanho original em caracteres.
 - Mostrar tamanho após a compressão.
 - Calcular e mostrar a taxa de compressão obtida.
 - Resultados.
 - Ao final, elaborar uma breve análise comparando:
 - Busca no cache versus busca no servidor.
 - Busca com e sem indexação.
 - Efeito da política LRU no gerenciamento do cache.
 - Comportamento das árvores *splay* na identificação de preferências e popularidade.
 - Redução do volume de dados obtida pela compressão de Huffman.
 - Os resultados devem evidenciar como diferentes estruturas de dados contribuem para desempenho, organização, personalização e eficiência de comunicação em um sistema de *streaming* simulado.

2. Representação gráfica da simulação.



3. Avaliação.

- O trabalho vale 70% da nota da 3ª unidade.
- Deve ser desenvolvido na linguagem Java usando orientação a objetos.
 - Data de entrega (única): 18/06/2026, somente via sigaa.
 - Formato de envio: seu-nome-pratica-off-3.zip.
- **O projeto é individual.**
 - Em caso de alguma dúvida, o(a) aluno(a) poderá ser chamado para esclarecimentos.
 - As estruturas de dados devem ser implementadas do zero.
- **Em caso de comprovação de cópia ou fraude, os trabalhos envolvidos receberão a nota ZERO.**

4. Roteiro de apresentação.

- Executar o programa resultante de cada enunciado, questão ou projeto.
- Para cada programa, execute uma operação por vez.
 - Durante a testagem de cada funcionalidade comente sobre os possíveis erros e exceções que podem ocorrer mediante entradas incorretas. Comente também sobre quais erros e exceções seu programa captura.
 - Cite as ferramentas e tecnologias utilizadas.
 - Seu programa implementa todas as funcionalidades requeridas?
- Após a demonstração de funcionamento, é hora de abrir o código. Explique:
 - Como o seu projeto está organizado? (se quiser, use uma representação gráfica para responder).
 - Quais as principais dificuldades e lições aprendidas com o desenvolvimento do projeto?
- Responda as perguntas detalhando o código que você implementou.
- Instruções para gravação:
 - O vídeo deve ter no máximo oito (8) minutos.
 - A gravação será o instrumento avaliativo.
 - Pontos de avaliação:
 - Sequência lógica e domínio do conteúdo (introdução, divisão organizada da apresentação, concatenação de ideias e conclusão).
 - Precisão na comunicação (colocação e entonação da voz, ritmo, dicção e linguagem).
 - Capacidade de argumentação.
 - Domínio e uso de material (organização no uso de recursos de apresentação).
- Sugestões de software para gravação:
 - Google Meet, Zoom, SimpleScreenRecorder, OBS, etc.
- Sobre o envio do vídeo:
 - O projeto deve ser armazenado e disponibilizado por meio de um link.
 - Exemplos: Google Drive, YouTube, Dropbox, etc.